3.d Functions - Inline Functions & Operator Overloading

Optimizing Higher-Order Functions & Custom Operators in Kotlin

Contents

1 **Inline Functions**

inline, noinline, crossinline, reified type parameters & non-local returns

2 **Operator Overloading**

Unary, binary, comparison, indexed access, invoke & augmented assignments

3 **Practice Exercises**

Solved and unsolved exercises combining inline functions & operator overloading

https://kotlinlang.org/docs/inline-functions.html

https://kotlinlang.org/docs/operator-overloading.html

Inline Functions

OVERVIEW

Higher-order functions use lambdas stored as **objects**, causing memory overhead and virtual calls. The **inline** keyword tells the compiler to copy the function body directly into the call site.

The Problem

- Each lambda is compiled to an anonymous class
- Object allocation on every invocation
- Captured variables add extra memory overhead
- Virtual method calls introduce runtime overhead

The Solution

- Compiler substitutes function body at call site
- No extra object allocation or virtual calls
- Lambdas passed to it are also inlined
- Best for small functions with lambda parameters

Non-local Returns & Inline Properties

Non-local Returns

A bare `return` inside a regular lambda is forbidden. But in an inline function lambda, it exits the enclosing function.

Example:

```
fun hasZeros(ints: List<Int>)
    : Boolean {
    ints.forEach {
        if (it == 0) return true
    }
    return false
}
```

forEach is inline, so return exits hasZeros

Inline Properties

The `inline` modifier can also be used on property accessors that don't have a backing field.

Individual accessors:

```
val foo: Foo
    inline get() = Foo()
```

Entire property:

```
inline var bar: Bar
    get() = ...
    set(v) { ... }
```

Accessors are inlined as regular inline functions at the call site.

noinline & crossinline

noinline

Function is passed as an object.
Prevents a specific lambda parameter from being inlined.

When to use:

- Passing a lambda to another non-inline function
- Storing a lambda in a field or variable
- Returning a lambda from the function

```
inline fun example(
    action: () -> Unit,
    noinline fallback: () -> Unit
) {
    // requires a real function object
    val list = listOf(fallback)
    println(list)
    list.forEach { it() }
    action()
}

example({ println("Action") }) { println("Fallback") }
```

```
Output:
[examples_3.S3C3InlineFunctionsKt$$Lambda/0x00…]
Fallback
Action
```

crossinline

A return inside a lambda performs a non-local return.
Crossline forbids non-local returns from the lambda.

When to use:

- Lambda called from a different context (e.g., local object or nested function)
- Passing lambda to a Runnable or callback

```
inline fun execute(crossinline block: () -> Unit) {
    val r = Runnable { block() }
    r.run()
}

execute {    // removed from execution – due to error
    return   // ❌ compiler error
}


execute {
    println("Hello")
    return@execute
}

println("continues normally")
```

```
Output:
Hello
continues normally
```

Reified Type Parameters

Kotlin erases generic type info at runtime (type erasure).
With **reified**, inline functions can access the actual type parameter at runtime — no reflection needed.

Without reified:

```
fun <T> TreeNode.findParentOfType(clazz: Class<T>): T? { ... }

// Verbose call site:
myTree.findParentOfType(MyTreeNode::class.java)
```

With reified:

```
inline fun <reified T> TreeNode.findParentOfType(): T? { ... }

// Clean call site — type passed as generic parameter:
myTree.findParentOfType<MyTreeNode>()
```

⚠ **Key rule:** Only inline functions can have reified type parameters. You can use `is T`, `as T`, and `T::class` at runtime.

Operator Overloading

OVERVIEW

Kotlin lets you provide custom implementations for predefined operators (+, -, *, etc.) on your types. Mark the corresponding function with the `operator` modifier.

```
data class Point(val x: Int, val y: Int) {
    operator fun plus(other: Point) =
        Point(x + other.x, y + other.y)
}
val p1 = Point(1, 2)
val p2 = Point(3, 4)
val p3 = p1 + p2
println(p3)
// Output: Point(x=4, y=6)

p1 + p2 is translated to p1.plus(p2) under the hood
```

Key Points

- Use `operator fun` with a specific function name
- Works as member or extension functions
- Only a predefined set of operators can be overloaded
- Optimized for basic types (no function call overhead)

Unary Operators

Operator	Function Name	Expression
+a	unaryPlus()	a.unaryPlus()
-a	unaryMinus()	a.unaryMinus()
!a	not()	a.not()
a++, a--	inc(), dec()	a.inc(), a.dec()

```
data class Point( val x: Int, val y: Int )

// extension function
operator fun Point.unaryMinus() = Point(-x, -y)

println(-Point(10, 20))
// Output: Point(x=-10, y=-20)
```

inc() & dec() Rules

- Must return a value (assigned to the variable)
- Should NOT mutate the object
- Return type must be a subtype of the receiver
- Prefix (++a): assigns first, then returns
- Postfix (a++): returns old value, then assigns

Binary Arithmetic Operators

Expression	Function Name	Translates to
a + b	plus	a.plus(b)
a - b	minus	a.minus(b)
a * b	times	a.times(b)
a / b	div	a.div(b)
a % b	rem	a.rem(b)

Augmented Assignments

For `a += b`, the compiler tries:

1. a.plusAssign(b) if available
2. a = a.plus(b) otherwise

If both exist, it's an ambiguity error!

```
operator fun Point.times(factor: Int): Point = Point(x * factor, y * factor)

val p = Point(1, 2)
println(p * 3)   // Output: Point(x=3, y=6)


Note: Operands don't have to be the same type. Overloaded operators are not always commutative (p * 3 works,
but 3 * p doesn't unless separately defined).

operator fun Int.times(p: Point): Point = Point(p.x * this, p.y * this)
```

Comparison, Indexed Access & Invoke

Expression	Function	Description
a == b	equals(b)	Structural equality
a > b, a < b	compareTo(b)	Returns Int (<0, 0, >0)
a in b	b.contains(a)	Containment check

Indexed Access

```
a[i]     → a.get(i)
a[i] = b → a.set(i, b)
```

Supports multiple index parameters

invoke() Operator

Parentheses translate to invoke calls:

```
a()    → a.invoke()
a(b)   → a.invoke(b)
```

This is why lambdas can be called with parentheses — they implement the invoke operator.

rangeTo & Iterator

```
a..b  → a.rangeTo(b)
a..<b → a.rangeUntil(b)
```

Destructuring:

```
val (x, y) = point  → component1(), component2()
```

Data classes generate these automatically

Best Practices & Summary

Inline Functions

- ✓ Use inline for small functions with lambda parameters
- ✓ Use reified to avoid passing Class<T> arguments explicitly
- ✗ Don’t inline large functions — generated code grows
- ✗ Don’t inline functions without lambda params or reified (compiler warns)
- ✓ Public inline functions cannot use private/internal APIs (use @PublishedApi)

Operator Overloading

- ✓ Maintain the original spirit of the operator
- ✓ Use operators when their meaning is clear and intuitive
- ✗ Don’t overuse — can make code confusing if misused
- ✓ Extension functions can add operators to third-party classes
- ✗ Cannot create arbitrary operators — only predefined set is overloadable

Practice Exercises

Exercise 1 — Inline Functions

SOLVED

Write an inline function `measureTime` that takes a lambda, executes it, measures execution time in milliseconds, and returns the elapsed time. Then use a `reified` function to filter a list by type.

Solution:

```
inline fun measureTime(block: () -> Unit): Long {
    val start = System.currentTimeMillis()
    block()
    return System.currentTimeMillis() - start
}

inline fun <reified T> List<Any>.filterByType(): List<T> =
    this.filter { it is T }.map { it as T }

val mixed = listOf(1, "hello", 2.5, "world", 3)
val strings = mixed.filterByType<String>()
println(strings)  // Output: [hello, world]
```

```
val mixed: List<Any> = buildList {
    repeat(500_000) { i ->
        add(i)
        add("text-$i")
        add(i.toDouble())
        add(i % 2 == 0)
    }
}

println("Number of elems  : ${"%7d".format(mixed.size)}")

var strings: List<String> = emptyList()

val elapsed = measureTime {
    strings = mixed.filterByType<String>()
}

println("Number of strings: ${"%7d".format(strings.size)}")
println("Elapsed time: ${elapsed} ms")
```

```
Output:
Number of elems  : 2000000
Number of strings:  500000
Elapsed time: 14 ms
```

- measureTime is inline → no lambda object allocated, the block body is copied to the call site
- reified T allows is T (instance of) and as T (type cast) checks at runtime without reflection — type info is preserved by inlining

Exercise 2 — Operator Overloading

SOLVED

Create a `Vector2D` data class with **x** and **y** (Double). Overload +, -, unaryMinus, times (scalar), and compareTo (by magnitude).

Solution:

```
data class Vector2D(val x: Double, val y: Double) : Comparable<Vector2D> {
    val magnitude get() = qrt(x * x + y * y)
    operator fun plus(other: Vector2D) = Vector2D(x + other.x, y + other.y)        // +
    operator fun minus(other: Vector2D) = Vector2D(x - other.x, y - other.y)       // Binary -
    operator fun unaryMinus() = Vector2D(-this.x, - this.y)                        // Unary -
    operator fun times(scalar: Double) = Vector2D(x * scalar, y * scalar)          // * n
    override fun compareTo(other: Vector2D) = magnitude.compareTo(other.magnitude) // <, >, <=, >=
}

val v1 = Vector2D(3.0, 4.0)
val v2 = Vector2D(1.0, 2.0)
println(v1 + v2)       // Outputs: Vector2D(x=4.0, y=6.0)
println(v1 * 2.0)      // Outputs: Vector2D(x=6.0, y=8.0)
println(v1 > v2)       // Outputs: true  // (5.0 > 2.24)
```

- Each `operator fun` maps to a symbolic operator. Implementing `Comparable` gives you <, >, <=, >= for free via `compareTo()`

Exercise 3 — Combined

SOLVED

Create a `Matrix2x2` class with indexed access (`get`/`set`), `plus`, and write an inline function `withMatrix` that applies a transform and prints the result.

Solution:

```
class Matrix2x2(private val data: Array<IntArray> = Array(2) { IntArray(2) }) {
    operator fun get(row: Int, col: Int) = data[row][col]
    operator fun set(row: Int, col: Int, value: Int) { data[row][col] = value }
    operator fun plus(other: Matrix2x2) = Matrix2x2( Array(2) { r -> IntArray(2) { c -> data[r][c] + other[r, c] } } )
    override fun toString() = data.joinToString("\n") { it.joinToString(", ") }
}

inline fun withMatrix(m: Matrix2x2, transform: (Matrix2x2) -> Matrix2x2) {
    println("Result:\n${transform(m)}")
}

val m1 = Matrix2x2().apply { this[0,0]=1; this[0,1]=2; this[1,0]=3; this[1,1]=4 }
val m2 = Matrix2x2().apply { this[0,0]=10; this[0,1]=20; this[1,0]=30; this[1,1]=40 }
println(m1)
println(m2)
withMatrix(m1) { it + m2 }
```

```
Output:
1, 2
3, 4
10, 20
30, 40
Result:
11, 22
33, 44
```

- `m[r, c]` uses the overloaded `get`/`set` operators (multi-index). The `inline` `withMatrix` avoids lambda overhead when chaining matrix operations.

Unsolved Exercises

TRY YOURSELF

4 Retry Inline Function

- Create an `inline fun <T> retry(times: Int, block: () -> T): T`
- Retries the block up to `times`, catching exceptions. Use `crossinline` for the block parameter
- Add a reified `onError` callback that filters by exception type

5 Money Class with Operator Overloading

- Create a `Money(amount: Double, currency: String)` class
- Overload `+`, `-`, `*` (scalar), `compareTo`, and `unaryMinus`
- Throw `IllegalArgumentException` when currencies differ

6 DSL Builder with Inline + Operators

- Build an `HtmlBuilder` DSL using inline functions with lambda receivers
- Overload `+String` to add text nodes and `invoke` to render output
- Combine both topics: use reified types to validate tag names at call-site